INSTITUTO FEDERAL DE MINAS GERAIS
ENGENHARIA DE COMPUTAÇÃO
TÉCNICAS DE PROGRAMAÇÃO
PROF. PATRICIA APARECIDA PROENCA AVILA

RELATÓRIO: IMPLEMENTAÇÃO DE GRAFOS EM LINGUAGEM C

Alunos: Paulo Henrique Ferreira
e Maria Eduarda Saldanha Alves

Bambuí – 2026

SUMÁRIO

INTRODUÇÃO **1**
IMPLEMENTAÇÃO **2**
Representação e organização da estrutura de dados 2
Decisões específicas de implementação 3
Operações de consulta e percurso 4
Grau e consultas sobre arestas 4
Carregamento e salvamento em arquivo 5
Especificações complementares 5
VALIDAÇÃO **6**
CONCLUSÃO **7**
REFERÊNCIAS BIBLIOGRÁFICAS **8**

INTRODUÇÃO

Grafos são modelos matemáticos usados para representar relações entre elementos, como as cidades de um mapa, os computadores de uma rede ou os estados de uma máquina de estados. Segundo Feofiloff, Kohayakawa e Wakabayashi (2004), um grafo pode ser definido por um conjunto de vértices e um conjunto de arestas que conectam pares desses vértices. Ele pode ser direcionado, quando as arestas têm sentido, ou não direcionado, quando não têm.

Este trabalho, desenvolvido na disciplina de Técnicas de Programação do curso de Engenharia da Computação do IFMG Campus Bambuí, trata do problema de representar grafos em um programa de forma reutilizável. Para isso, foi implementado em linguagem C um Tipo Abstrato de Dados (TAD) Grafo, reunido em um módulo (`grafo.h` e `grafo.c`) que oferece operações para criar e destruir grafos, inserir vértices e arestas, consultá-los e percorrê-los, e salvar e carregar um grafo em arquivo de texto.

A TAD guarda apenas a estrutura do grafo, com identificadores inteiros sequenciais para vértices e arestas, e admite arestas múltiplas e laços. O mesmo módulo atende grafos dirigidos e não dirigidos: o grafo é criado da mesma forma, e o que muda são as operações usadas para consultá-lo, como as estrelas de entrada e de saída, no caso dos dirigidos. Junto ao módulo, foi desenvolvido um programa de testes interativo que permite exercitar cada operação.

IMPLEMENTAÇÃO

Representação e organização da estrutura de dados

A implementação do TAD Grafo foi realizada em linguagem C e organizada em dois arquivos principais: `grafo.h`, responsável pela definição da estrutura e pela declaração das funções, e `grafo.c`, que contém a implementação dessas funções.

A estrutura `grafo` reúne as informações necessárias para representar o grafo. Ela possui os campos `v` e `a`, que indicam as capacidades máximas de vértices e arestas, respectivamente, e os campos `V` e `A`, que armazenam as quantidades atuais desses elementos. Além disso, são utilizados dois vetores: `vertices`, vetor de inteiros destinado ao armazenamento dos identificadores dos vértices, e `arestas`, vetor de estruturas aresta. Cada elemento desse vetor representa uma aresta e guarda seus dois extremos nos campos `alfa`, que armazena o vértice de partida, e `omega`, que armazena o vértice de chegada.

Os vetores são alocados dinamicamente durante a criação do grafo, de acordo com as capacidades informadas. Essa organização permite reservar memória para os elementos que poderão ser inseridos e controlar a quantidade de vértices e arestas existentes por meio dos campos `V` e `A`.

Para ilustrar a representação adotada, considera-se um grafo com três vértices e três arestas, conforme apresentado na Figura 1. A figura 2 relaciona a representação gráfica do grafo à organização dos elementos nos vetores utilizados na implementação.

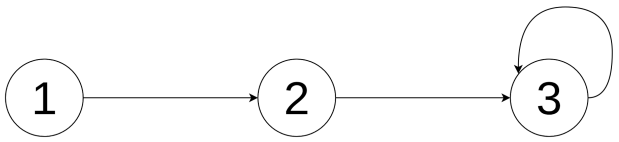


Figura 1 – Representação gráfica de um grafo.

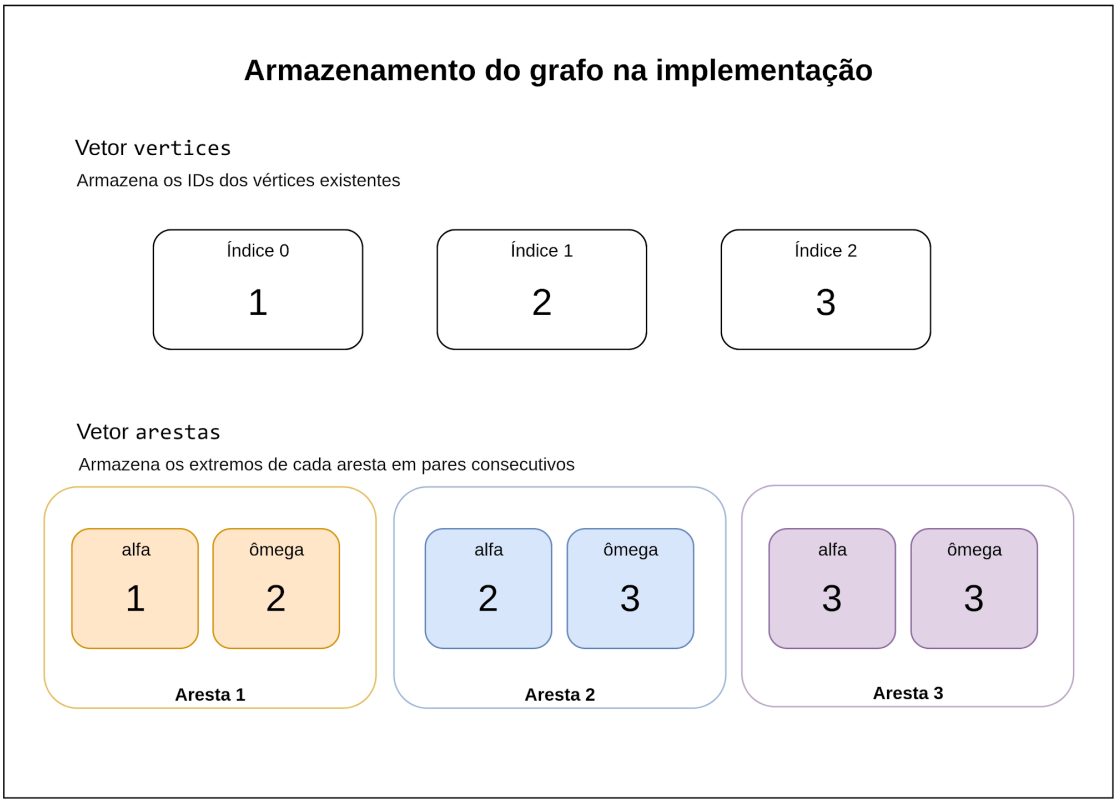


Figura 2 – Representação de armazenamento de seus vértices e arestas.

Decisões específicas de implementação

Os identificadores dos vértices e das arestas são sequenciais. Os vértices são armazenados no vetor `vertices`, enquanto cada aresta é um elemento do vetor `arestas`, que guarda o vértice de partida no campo `alfa` e o vértice de chegada no campo `omega`. Assim, o identificador de uma aresta corresponde à sua posição no vetor mais um, já que os identificadores começam em 1 e as posições do vetor começam em 0.

A função `GVproximoVertice` utiliza a sequência dos identificadores para localizar o próximo vértice. Como a implementação não possui uma operação de remoção de vértices, os identificadores permanecem sequenciais. Dessa forma, a função pode verificar diretamente se existe um identificador posterior ao informado, sem precisar percorrer todo o vetor de vértices.

De maneira semelhante, a função `GAprimeiraAresta` considera que, se houver arestas no grafo, a primeira possui identificador 1. Essa decisão decorre da atribuição sequencial dos identificadores durante a criação das arestas.

As funções de criação verificam se a capacidade máxima foi atingida antes de inserir novos elementos. Quando não há espaço disponível, retornam zero para indicar que a operação não foi realizada. Nas consultas que dependem de um grafo válido, o tratamento de ponteiros nulos também é utilizado para evitar o acesso à estrutura quando ela não está disponível.

Operações de consulta e percurso

As operações de consulta permitem verificar a existência de vértices e arestas, obter identificadores e consultar as quantidades atuais e máximas de elementos. As funções responsáveis por essas operações utilizam as informações armazenadas na estrutura e, quando necessário, percorrem os vetores para localizar os elementos desejados.

A verificação de arestas considera a ordem dos extremos conforme a operação realizada. Para a consulta de uma aresta não dirigida, os extremos podem ser informados em qualquer ordem. Já para uma aresta dirigida, é necessário respeitar a ordem de partida e chegada armazenada no vetor. As funções de consulta também permitem obter o identificador de uma aresta a partir de seus extremos.

O TAD disponibiliza operações para percorrer os vértices e as arestas, além de localizar arestas incidentes a um vértice. Para grafos dirigidos, existem operações específicas para consultar as arestas que chegam a um vértice e as que partem dele. Essas buscas são realizadas por meio da análise dos extremos de cada aresta armazenada.

Grau e consultas sobre arestas

A função `GIpegaGrau` calcula o grau de um vértice percorrendo as arestas do grafo e verificando se ele aparece em algum dos extremos de cada aresta. Quando uma aresta é um laço, ela é contabilizada duas vezes, pois o mesmo vértice aparece tanto no campo `alfa` quanto no campo `omega`.

As funções `GAprimaAresta` e `GAproxAresta` permitem localizar a primeira aresta incidente a um vértice e a próxima aresta incidente, respectivamente. Para grafos dirigidos, as funções `GAprimaEntrada`, `GAproxEntrada`, `GAprimaSaida` e `GAproxSaida` realizam buscas considerando o campo `omega` (vértice de chegada) ou o campo `alfa` (vértice de partida), conforme a operação.

Para consultar informações específicas de uma aresta, a função `GBarestaLaco` verifica se seus extremos são iguais. As funções `GAlfa` e `GOmega` retornam, respectivamente, o vértice de partida e o vértice de chegada. Já a função

`GVvizinho` recebe uma aresta e um vértice e retorna o outro extremo quando o vértice informado pertence à aresta. Caso contrário, retorna zero.

Carregamento e salvamento em arquivo

A implementação também permite carregar e salvar grafos em arquivos de texto. Para o carregamento, a primeira linha do arquivo contém as quantidades de vértices e arestas. Nas linhas seguintes, são informados os pares de extremos que representam as arestas. A função `GGcarregaGrafo` realiza a leitura dessas informações, cria os vértices e registra as arestas na estrutura.

A função `GBsalvaGrafo` realiza o processo inverso, gravando no arquivo as quantidades atuais de vértices e arestas, seguidas pelos pares de extremos de cada aresta. As operações verificam a ocorrência de erros durante a abertura, a leitura ou a gravação do arquivo e retornam um indicador de sucesso ou falha.

Especificações complementares

Uma decisão considerada na implementação foi a utilização do valor zero para indicar que determinadas operações não foram realizadas ou não encontraram um resultado. Como os identificadores válidos de vértices e arestas começam em 1, esse valor pode ser utilizado como indicador de ausência de resultado nas operações que retornam identificadores.

Outra decisão foi o tratamento dos laços no cálculo do grau: quando uma aresta liga um vértice a ele mesmo, ela é contada duas vezes. Além disso, os identificadores sequenciais permitem localizar o próximo vértice e a primeira aresta sem a necessidade de percorrer integralmente os respectivos vetores.

VALIDAÇÃO

A validação foi realizada por meio de testes das funções implementadas para o Tipo Abstrato de Dados (TAD) Grafo. Para auxiliar nesse processo, foi desenvolvida uma função *main* específica para executar as operações e observar seus retornos, permitindo verificar o comportamento das funcionalidades durante o desenvolvimento.

Foram testadas operações de consulta e percurso de vértices e arestas, incluindo a obtenção do primeiro elemento e do próximo elemento disponível. Também foram considerados casos em que não há elementos a serem retornados, como quando não existe um próximo vértice ou aresta.

Também foram testadas as operações que dependem dos extremos das arestas, como a verificação de arestas dirigidas e não dirigidas, com os extremos informados nas duas ordens, e a obtenção dos identificadores a partir dos extremos. Foi testada a criação de um laço, verificando o grau do vértice (contado duas vezes), as funções `GBarestaLaco`, `GAlfa`, `GOmega` e `GVvizinho`, e as buscas de arestas incidentes, de entrada e de saída. Foram verificados ainda os limites de capacidade, com a recusa de novos vértices e arestas quando o máximo é atingido, e o salvamento de um grafo em arquivo seguido de seu carregamento, conferindo se as quantidades e as arestas se mantinham iguais às do grafo original.

Durante o desenvolvimento, foram adicionadas verificações de segurança para tratar determinadas situações inválidas, como a utilização de ponteiros nulos em funções que realizam consultas ao grafo. Também foi adicionada a verificação dos vértices informados na criação de uma aresta, que passa a ser recusada quando algum dos extremos não existe. Além disso, os identificadores dos vértices e das arestas foram definidos sequencialmente, a partir do valor 1, seguindo a ordem de criação dos elementos.

Os testes realizados auxiliaram na verificação do comportamento das funções e na identificação de ajustes necessários durante a implementação.

CONCLUSÃO

Durante o desenvolvimento do trabalho, a principal dificuldade encontrada foi o raciocínio inicial para estruturar a TAD Grafo: decidir como representar e armazenar os vértices e as arestas e como organizar as operações a serem implementadas. Essa dificuldade foi superada ao seguir os identificadores sequenciais definidos na especificação e ao guardar as arestas em um único vetor de estruturas, em que cada elemento representa uma aresta e guarda seus vértices de partida e de chegada nos campos `alfa` e `omega`. Com essa estrutura estabelecida, as demais operações passaram a seguir uma lógica semelhante, o que tornou o desenvolvimento mais claro e permitiu avançar gradualmente na construção e nos testes.

Ao longo da implementação, aprofundamos a compreensão da alocação dinâmica de memória em C, com o uso de `malloc` para reservar e de `free` para liberar a memória dos vetores e da estrutura do grafo, além da manipulação de arquivos de texto para salvar e carregar grafos. O trabalho também permitiu praticar a modularização, separando a interface da TAD (`grafo.h`) de sua implementação (`grafo.c`).

Ao final, a TAD atende às operações da especificação e funciona tanto para grafos dirigidos quanto para não dirigidos. Há pontos que poderiam ser melhorados. O primeiro é o vetor de vértices, que guarda apenas os números 1, 2, 3 e assim por diante: como o vértice de número `i` sempre tem identificador `i`, esse vetor não armazena nenhuma informação nova e poderia ser substituído apenas pela contagem de vértices, verificando se um vértice existe pelo intervalo (entre 1 e a quantidade de vértices criados). O segundo são as buscas pelas arestas, que percorrem o vetor inteiro a cada consulta. Isso poderia ser melhorado com listas de adjacência, em que cada vértice guarda a lista das suas próprias arestas, de modo que a consulta percorra apenas as arestas do vértice em questão, e não todas as do grafo.

REFERÊNCIAS BIBLIOGRÁFICAS

SCHILDT, Herbert. *C, completo e total*. Tradução e revisão técnica de Roberto Carlos Mayer. 3. ed. rev. e atual. São Paulo: Makron Books, 1997.

COHEN, Marcelo. Modularização em C. Porto Alegre: Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS), mar. 2013. Disponível em: *https://www.inf.pucrs.br/flash/progswb/present/U01_progC/makefiles/handout.html*. Acesso em: 21 set. 2026.

UNIVERSITAT POLITÈCNICA DE VALÈNCIA. Curso Teoría básica de grafos y análisis de 4 conocidos problemas. YouTube, jun. 2019. Playlist (43 vídeos). Disponível em: *https://www.youtube.com/playlist?list=PL5098BF5A01819B3B*. Acesso em: 13 out. 2026.

CRUZ, Adriano Joaquim de Oliveira. **Curso de linguagem C**. Rio de Janeiro: Universidade Federal do Rio de Janeiro, Núcleo de Computação Eletrônica, 2011. Disponível em: https://drive.google.com/file/d/1-74hvZLnP1DcSO8dk5HUFxaPNUH47jW_/view. Acesso em: 18 out. 2026.

FEOFILOFF, Paulo; KOHAYAKAWA, Yoshiharu; WAKABAYASHI, Yoshiko. **Uma introdução sucinta à teoria dos grafos**. 2004. Disponível em: https://www.ime.usp.br/~pf/teoriadosgrafos/. Acesso em: 18 out. 2026.